Relatório de Bugs

Aplicação: Automation Exercise — https://www.automationexercise.com

DOCUMENTO	Relatório de defeitos e propostas de correção
VERSÃO	1.0
DATA	14/10/2026
AUTOR	Angelo Roque — Análise de Qualidade
FERRAMENTAS	REA 6.4.0 (captura de cenários de navegador e evidências), Chromium 1243 (Playwright), cURL, Cypress 15.20.0
STATUS GERAL	5 defeitos confirmados e reproduzíveis — 1 alta, 2 média, 2 baixa

1. Sumário executivo

Este relatório apresenta os defeitos identificados na aplicação web Automation Exercise durante uma análise exploratória apoiada por engenharia reversa da aplicação em execução. Foram confirmados **cinco defeitos**, todos reproduzíveis e cobertos por testes automatizados em Cypress que falham enquanto o defeito persistir.

O defeito mais crítico (**BUG-01**) permite adicionar produtos ao carrinho com **quantidade negativa**, o que gera valores totais negativos. Em um sistema de comércio eletrônico real, essa falha pode corromper o valor do pedido e permitir manipulação de preço. Os demais defeitos envolvem validação de quantidade zero, recursos carregados de forma insegura (Mixed Content) e incoerências no contrato HTTP da API pública.

ID	Título	Área	Severidade	Prioridade
BUG-01	Carrinho aceita quantidade negativa	Carrinho / Back-end	**Alta**	P1
BUG-02	Carrinho aceita quantidade zero	Carrinho / Back-end	**Média**	P2
BUG-03	Fontes carregadas via HTTP em página HTTPS (Mixed Content)	Front-end / CSS	**Baixa**	P3
BUG-04	API retorna HTTP 200 para respostas de erro	API REST	**Média**	P2
BUG-05	API serve JSON com Content-Type text/html	API REST	**Baixa**	P3

2. Escopo e metodologia

2.1 Escopo analisado

- Página inicial (carregamento de recursos, console e rede);
- Página de detalhes do produto e fluxo de adição ao carrinho;
- Página do carrinho (`/view_cart`);
- API pública documentada em `/api_list`.

Fora do escopo: cadastro, login pela interface, checkout, pagamento, formulário *Contact Us* (upload de arquivo) e testes de carga ou de segurança ofensivos.

2.2 Metodologia

1. **Captura de cenários reais com a REA:** o comando `rea capture-browser-scenario` executou cenários declarativos em um Chromium isolado (perfil temporário, sem dados do usuário) e registrou como evidência os eventos de console, os erros de página, as requisições e respostas de rede e a árvore de acessibilidade final.
2. **Análise estática dos recursos:** inspeção dos arquivos `/static/js/cart.js` e `/static/css/main.css` e da marcação HTML, para localizar a causa raiz.
3. **Verificação da API:** requisições com cURL comparando o status HTTP real, o campo `responseCode` do corpo e os cabeçalhos.
4. **Confirmação automatizada:** cada defeito ganhou um teste Cypress que descreve o comportamento correto. Todos os testes falharam, o que confirma os defeitos.

2.3 Ambiente de teste

Sistema operacional	Ubuntu 26.04.1 LTS (linux-x64)
Navegador	Chromium 1243 (Playwright), headless, 1280×720, locale en-US, fuso UTC
Node.js	v24.14.0
Cypress	15.20.0 (navegador Electron)
Tecnologia observada no servidor	Django (inferido pelo token `csrfmiddlewaretoken`), Phusion Passenger 6.2.0, atrás do Cloudflare

2.4 Critérios de classificação

Severidade	Critério
Alta	Afeta integridade de dados financeiros ou regras de negócio centrais; pode ser explorado pelo usuário.
Média	Comportamento incorreto com impacto funcional ou de integração, sem perda financeira direta.
Baixa	Impacto visual, de conformidade ou de boas práticas, com contorno simples.

3. Detalhamento dos defeitos

BUG-01 — Carrinho aceita quantidade negativa

Severidade / Prioridade	**Alta** / P1
Componente	`/product_details/{id}`, `/static/js/cart.js`, endpoint `GET /add_to_cart/{id}`
Reprodutibilidade	100% (determinística)
Pré-condição	Nenhuma (usuário anônimo, carrinho vazio)

PASSOS PARA REPRODUZIR

1. Acessar `https://www.automationexercise.com/product_details/1` (produto *Blue Top*, Rs. 500).
2. No campo **Quantity**, apagar o valor e digitar `-3`.
3. Clicar em **Add to cart**.
4. No modal, clicar em **View Cart**.

RESULTADO ESPERADO

A quantidade deve ser rejeitada, com mensagem de validação (mínimo 1). Nenhum item deve ser adicionado ao carrinho.

RESULTADO OBTIDO

O item é adicionado e o carrinho exibe: `Blue Top | Rs. 500 | -3 | Rs. -1500`.

EVIDÊNCIAS

- **Rede (REA):** requisição `GET https://www.automationexercise.com/add_to_cart/1?quantity=-3` aceita pelo servidor.

- **Árvore de acessibilidade do carrinho (REA):** `row "Product Image Blue Top Women > Tops Rs. 500 -3 Rs. -1500"`.
- **Cypress:** `AssertionError: expected -3 to be above 0`.

ANÁLISE DE CAUSA RAIZ

O campo é declarado como `<input type="number" id="quantity" min="1">`. O atributo `min`, porém, só é aplicado pelo navegador quando um formulário é **enviado**. O botão é `<button type="button" class="cart">`, e o envio é feito por AJAX em `cart.js`, que lê o valor bruto e o envia sem nenhuma validação:

```
$('.cart').on('click', function() {
    var product = document.getElementById("product_id").value;
    var quantity = document.getElementById("quantity").value;
    $.ajax({
        url: "/add_to_cart/" + product,
        data: {'quantity': quantity},
        ...
```

O servidor também aceita o valor sem validar o intervalo. A falha existe, portanto, em **duas camadas**. A camada decisiva é o back-end, porque a validação no navegador pode ser contornada chamando a URL diretamente.

IMPACTO

- Total do carrinho negativo ou reduzido indevidamente (combinando itens positivos e negativos);
- Risco de manipulação de preço no checkout e de inconsistência em estoque, relatórios e faturamento;
- Experiência do usuário degradada e falta de confiabilidade nos valores exibidos.

SOLUÇÃO PROPOSTA

1. Back-end (obrigatório): validar a quantidade na view, como inteiro dentro de um intervalo permitido, e responder com erro 400 quando o valor for inválido. Exemplo em Django:

```
from django.http import JsonResponse

MIN_QUANTITY = 1
MAX_QUANTITY = 99

def add_to_cart(request, product_id):
    try:
        quantity = int(request.GET.get("quantity", MIN_QUANTITY))
    except (TypeError, ValueError):
        return JsonResponse({"message": "Quantidade inválida."}, status=400)

    if not MIN_QUANTITY <= quantity <= MAX_QUANTITY:
        return JsonResponse(
            {"message": f"A quantidade deve estar entre {MIN_QUANTITY} e
{MAX_QUANTITY}."},
            status=400,
        )
```

```
    # ... lógica existente de adição ao carrinho ...
```

2. Modelo de dados (defesa em profundidade): impedir que valores inválidos sejam persistidos, mesmo que outra rota grave no carrinho:

```
from django.db import models

class CartItem(models.Model):
    quantity = models.PositiveIntegerField()

    class Meta:
        constraints = [
            models.CheckConstraint(
                condition=models.Q(quantity__gte=1),
                name="cart_item_quantity_gte_1",
            ),
        ]
```

3. Front-end (experiência do usuário): usar a validação nativa do navegador antes de enviar e tratar o erro do servidor:

```
$('.cart').on('click', function () {
    var input = document.getElementById("quantity");
    var quantity = Number(input.value);

    if (!input.checkValidity() || !Number.isInteger(quantity) || quantity < 1) {
        input.reportValidity();
        return;
    }

    $.ajax({
        url: "/add_to_cart/" + document.getElementById("product_id").value,
        data: { quantity: quantity },
        success: function () {
            $('#cartModal').removeClass('fade').addClass('show');
        },
        error: function (xhr) {
            alert(xhr.responseJSON?.message || "Não foi possível adicionar o produto.");
        }
    });
});
```

4. Saneamento: executar uma rotina única para remover ou corrigir itens de carrinho já gravados com quantidade menor ou igual a zero.

CRITÉRIO DE ACEITE

- `GET /add_to_cart/1?quantity=-3` retorna HTTP 400 e não altera o carrinho;
- A interface exibe a mensagem de validação e não abre o modal de sucesso;
- O teste Cypress “*BUG-01: não deve aceitar quantidade negativa (-3)*” passa.

BUG-02 — Carrinho aceita quantidade zero

Severidade / Prioridade	**Média** / P2
Componente	Mesmo do BUG-01
Reprodutibilidade	100%

PASSOS PARA REPRODUZIR

Os mesmos do BUG-01, informando `0` no campo **Quantity**.

RESULTADO ESPERADO

A quantidade deve ser rejeitada; nenhum item deve ser adicionado.

RESULTADO OBTIDO

O carrinho exibe uma linha sem valor: `Blue Top | Rs. 500 | 0 | Rs. 0`.

EVIDÊNCIAS

- **REA:** árvore de acessibilidade com `row "Product Image Blue Top Women > Tops Rs. 500 0 Rs. 0"`.
- **Cypress:** `AssertionError: expected 0 to be above 0`.

CAUSA RAIZ E SOLUÇÃO

A causa raiz é a mesma do BUG-01: falta validar o limite inferior. A correção proposta para o BUG-01 (intervalo `1..99` no back-end, restrição no banco e validação no front-end) **resolve os dois defeitos**. Recomenda-se corrigi-los na mesma entrega.

IMPACTO

Itens "fantasmas" no carrinho, que confundem o usuário e podem gerar pedidos com linhas sem valor e falhas em integrações de estoque e logística.

BUG-03 — Fontes carregadas via HTTP em página HTTPS (Mixed Content)

Severidade / Prioridade	**Baixa** / P3
Componente	`/static/css/main.css`, linhas 5 a 7
Reprodutibilidade	100% em navegadores modernos

PASSOS PARA REPRODUZIR

1. Abrir `https://www.automationexercise.com/` com as ferramentas de desenvolvedor abertas (aba **Console**).
2. Observar os erros de *Mixed Content*.

RESULTADO ESPERADO

Todos os recursos são carregados via HTTPS e as fontes Roboto, Open Sans e Abel são aplicadas.

RESULTADO OBTIDO

O navegador bloqueia três folhas de estilo do Google Fonts, e a página é exibida com fontes substitutas.

EVIDÊNCIAS

- **Console (REA):** `Mixed Content: The page at 'https://www.automationexercise.com/' was loaded over HTTPS, but requested an insecure stylesheet 'http://fonts.googleapis.com/css?family=Abel'. This request has been blocked; the content must be served over HTTPS.` (mesma mensagem para Roboto e Open Sans);
- **Rede (REA):** três eventos `request-failed` para `http://fonts.googleapis.com/...`;
- **Código-fonte:**

```
@import url(http://fonts.googleapis.com/css?
family=Roboto:400,300,400italic,500,700,100);
@import url(http://fonts.googleapis.com/css?family=Open+Sans:400,800,300,600,700);
@import url(http://fonts.googleapis.com/css?family=Abel);
```

- **Cypress:** `@import http:// em .../static/css/main.css: expected [ Array(3) ] to be empty`.

CAUSA RAIZ

As URLs foram escritas com o esquema `http://`, provavelmente desde antes da migração do site para HTTPS. Os navegadores bloqueiam esse tipo de recurso em páginas seguras.

IMPACTO

Identidade visual diferente da projetada, erros no console que poluem a monitoração e prejuízo em auditorias de qualidade e segurança (Lighthouse, por exemplo). Além disso, os `@import` encadeados atrasam a renderização.

SOLUÇÃO PROPOSTA

1. Correção mínima: trocar o esquema para HTTPS em `main.css`:

```
@import url(https://fonts.googleapis.com/css?family=Roboto:400,300,400italic,500,700,100);
@import url(https://fonts.googleapis.com/css?family=Open+Sans:400,800,300,600,700);
@import url(https://fonts.googleapis.com/css?family=Abel);
```

2. Correção recomendada (melhor desempenho): remover os `@import` do CSS e carregar as fontes em uma única requisição no `<head>` do template base:

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link rel="stylesheet"
      href="https://fonts.googleapis.com/css2?
family=Abel&family=Open+Sans:wght@300;400;600;700;800&family=Roboto:ital,wght@0,100;0,300;
0,400;0,500;0,700;1,400&display=swap">
```

3. Prevenção: adicionar o cabeçalho `Content-Security-Policy: upgrade-insecure-requests`, que faz o navegador atualizar automaticamente para HTTPS os recursos referenciados com HTTP.

CRITÉRIO DE ACEITE

Nenhum erro de *Mixed Content* no console da página inicial; fontes aplicadas; teste Cypress do BUG-03 passando.

BUG-04 — API retorna HTTP 200 para respostas de erro

Severidade / Prioridade	**Média** / P2
Componente	Endpoints `/api/*`
Reprodutibilidade	100%

PASSOS PARA REPRODUZIR

```
curl -i -X POST https://www.automationexercise.com/api/productsList
```

RESULTADO ESPERADO

Status HTTP coerente com o erro (`405 Method Not Allowed`, com o cabeçalho `Allow: GET`).

RESULTADO OBTIDO

Status HTTP `200 OK`, com o erro apenas no corpo: `{"responseCode": 405, "message": "This request method is not supported."}`.

EVIDÊNCIAS

Requisição	Status HTTP real	responseCode no corpo
`POST /api/productsList`	200	405
`PUT /api/brandsList`	200	405
`DELETE /api/verifyLogin`	200	405
`POST /api/searchProduct` (sem parâmetro)	200	400
`POST /api/verifyLogin` (parâmetros em query string)	200	400

Cypress: `status HTTP real: expected 200 to equal 405` e `expected 200 to equal 400`.

CAUSA RAIZ

As views da API devolvem o código de erro como dado no corpo JSON, mas não o definem como status da resposta HTTP, que fica no padrão 200. A página `/api_list` documenta “Response Code: 405” sem esclarecer que o código não é o status HTTP.

IMPACTO

- Clientes HTTP, gateways, CDNs, ferramentas de monitoramento (APM) e alertas tratam erros como sucesso;
- Bibliotecas que lançam exceção em 4xx/5xx (fetch com `response.ok`, axios, Cypress `cy.request`) não detectam a falha;
- Respostas de erro podem ser armazenadas em cache como respostas válidas.

SOLUÇÃO PROPOSTA

Centralizar a construção das respostas da API em um único utilitário, que define o status HTTP e **mantém o campo `responseCode` no corpo** por compatibilidade com os clientes atuais:

```
from django.http import JsonResponse
from django.views.decorators.http import require_http_methods

def api_response(status, **payload):
    return JsonResponse({"responseCode": status, **payload}, status=status)

def method_not_allowed(allowed):
    response = api_response(405, message="This request method is not supported.")
    response["Allow"] = ", ".join(allowed)
    return response

def products_list(request):
    if request.method != "GET":
        return method_not_allowed(["GET"])
    return api_response(200, products=serialize_products())

def search_product(request):
    if request.method != "POST":
        return method_not_allowed(["POST"])
    term = request.POST.get("search_product")
    if not term:
        return api_response(
            400, message="Bad request, search_product parameter is missing in POST
request."
        )
    return api_response(200, products=search(term))
```

Compatibilidade: o corpo das respostas não muda; muda apenas o status HTTP. Por isso, a correção deve ser anunciada na página `/api_list`, deixando claro que “Response Code” passa a valer também como status HTTP. Clientes que usam `failOnStatusCode` ou equivalente precisarão tratar respostas 4xx.

CRITÉRIO DE ACEITE

Para todos os casos da tabela de evidências, o status HTTP é igual ao `responseCode` do corpo, e as respostas 405 incluem o cabeçalho `Allow`.

BUG-05 — API serve JSON com Content-Type text/html

Severidade / Prioridade	**Baixa** / P3
Componente	Endpoints `/api/*`
Reprodutibilidade	100%

PASSOS PARA REPRODUZIR

```
curl -s -D - -o /dev/null https://www.automationexercise.com/api/productsList
curl -s -I https://www.automationexercise.com/api/productsList
```

RESULTADO ESPERADO

`Content-Type: application/json` em todas as respostas da API.

RESULTADO OBTIDO

- `GET` e `POST`: `content-type: text/html; charset=utf-8`, com corpo JSON;
- `HEAD` da mesma URL: `content-type: application/json`. O tipo informado no `HEAD` diverge do `GET`.

Cypress: `expected 'text/html; charset=utf-8' to include 'application/json'`.

CAUSA RAIZ (HIPÓTESE)

O comportamento é compatível com respostas montadas como `HttpResponse(json.dumps(...))`, que usa `text/html` por padrão no Django. A divergência no `HEAD` sugere que esse método é tratado por outro caminho (outra camada da aplicação ou da infraestrutura). Como o código do servidor não está disponível, essa causa precisa ser confirmada pela equipe de desenvolvimento.

IMPACTO

Clientes que dependem do `Content-Type` não convertem o corpo automaticamente em JSON (por exemplo, no Cypress, `res.body` chega como texto). O cabeçalho `X-Content-Type-Options: nosniff`, também presente, impede que o navegador corrija o tipo. Além disso, a resposta fica mais exposta a interpretação como HTML.

SOLUÇÃO PROPOSTA

Usar `JsonResponse` em todas as respostas da API. O utilitário `api_response` proposto no BUG-04 já faz isso, então **os dois defeitos podem ser corrigidos na mesma entrega**. Recomenda-se também confirmar que `GET` e `HEAD` passam pela mesma view, para que retornem cabeçalhos idênticos.

CRITÉRIO DE ACEITE

`GET`, `POST` e `HEAD` retornam `Content-Type: application/json`; teste Cypress do BUG-05 passando.

4. Plano de correção recomendado

Entrega	Defeitos	Ações	Esforço estimado
1 — Urgente	BUG-01, BUG-02	Validação de quantidade no back-end, restrição no banco, validação e tratamento de erro no front-end, saneamento de dados	Baixo a médio
2	BUG-04, BUG-05	Utilitário único de resposta da API com status e Content-Type corretos; atualização da documentação	Baixo
3	BUG-03	Fontes via HTTPS no `<head>`; cabeçalho CSP `upgrade-insecure-requests`	Muito baixo

> *As estimativas são qualitativas e baseadas apenas no comportamento observado. O código do servidor não foi analisado diretamente.*

5. Recomendações adicionais

- **Operações que alteram estado via GET:** `/add_to_cart/{id}` e `/delete_cart/{id}` alteram o carrinho por meio de requisições GET. Recomenda-se migrá-las para `POST`/`DELETE` com proteção CSRF, porque requisições GET podem ser disparadas por links, pré-carregamento do navegador ou rastreadores.
- **Regressão automatizada:** os testes Cypress deste relatório estão em `cypress/e2e/known-bugs/` do repositório e rodam diariamente no pipeline de integração contínua, em um job separado da suíte principal. Eles falham hoje e devem passar após as correções.
- **Monitoramento de console:** adicionar verificação automática de erros de console (Mixed Content, exceções JavaScript) nos testes de fumaça.
- **Contrato da API:** publicar uma especificação OpenAPI com status HTTP, cabeçalhos e esquemas de resposta para cada endpoint.

6. Rastreabilidade com os testes automatizados

Defeito	Teste Cypress	Resultado atual
BUG-01	[BUG-01] não deve aceitar quantidade negativa (-3)	Falha — `expected -3 to be above 0`
BUG-02	[BUG-02] não deve aceitar quantidade zero (0)	Falha — `expected 0 to be above 0`
BUG-03	[BUG-03] as folhas de estilo não devem importar recursos via http://	Falha — 3 imports inseguros em `main.css`
BUG-04	[BUG-04] POST /api/productsList deve retornar HTTP 405	Falha — `expected 200 to equal 405`
BUG-04	[BUG-04] PUT /api/brandsList deve retornar HTTP 405	Falha — `expected 200 to equal 405`
BUG-04	[BUG-04] DELETE /api/verifyLogin deve retornar HTTP 405	Falha — `expected 200 to equal 405`
BUG-04	[BUG-04] POST /api/searchProduct sem parâmetro deve retornar HTTP 400	Falha — `expected 200 to equal 400`
BUG-04	[BUG-04] POST /api/verifyLogin com parâmetros na query string deve retornar HTTP 400	Falha — `expected 200 to equal 400`
BUG-05	[BUG-05] GET /api/productsList deve declarar Content-Type JSON	Falha — `expected 'text/html; charset=utf-8' to include 'application/json'`

7. Limitações

- A análise foi feita de fora (caixa-preta e caixa-cinza): as causas raiz no servidor são inferidas a partir do comportamento observado, do HTML, do CSS e do JavaScript públicos. Os trechos de código do back-end são propostas ilustrativas, e não cópias do código existente.
- Os resultados refletem o estado do site em 14/10/2026. Por ser um ambiente público e compartilhado, o comportamento pode mudar sem aviso.
- As áreas listadas como fora do escopo (seção 2.1) não foram avaliadas, e a ausência de defeitos nelas não está comprovada.